

AUDISANKARA INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)

Capstone Project Report

on

**IOT BASED AUTOMATIC VEHICLE ACCIDENT DETECTION
AND RESCUE SYSTEM**

Submitted By

N.V. MAHITHA

232H5A0413

T. SWATHI

222H1A04K4

V.SANTHOSH

222H1A04M2

SK. LAHUR

222H1A04I5

Under the esteemed guidance of

Dr.P.Sree Lakshmi, M.Tech,Ph.D

Associate Professor in the Department of

ELECTRONICS & COMMUNICATION ENGINEERING



AUDISANKARA COLLEGE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

**Accredited by NAAC with 'A+' Grade by AICTE | Affiliated to JNTUA
NH5 Bypass Road, Gudur – 524101, Nellore (DT.), Andhra Pradesh**

www.audisankara.ac.in

AUDISANKARA INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

Accredited by NAAC with 'A+' Grade Approved by AICTE | Affiliated to JNTUA

NH5 Bypass Road, Gudur – 524101, Nellore (DT.), Andhra Pradesh

DEPARTMENT OF

ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certify that the Capstone project report on entitled **IOT BASED AUTOMATIC VEHICLE ACCIDENT DETECTION AND RESCUE SYSTEM** is the bonafide work done By **N.V.MAHITHA(232H5A0413),T.SWATHI(222H1A04K4),V.SANTHOSH(222H1A04M2), SK.LAHUR (222H1A04I5)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in ELECTRONICS&COMMUNICATION ENGINEERING**, from Jawaharlal Nehru Technological University Anantapur, Anantapuramu, during the year **2025- 2026**

Project Guide

Dr P Sree Lakshmi,M.Tech.,Ph.D

Associate Professor,

Department of ECE,

ASIT, GUDUR-NELLORE(DT).

Head of the Department

Dr. Kishore Mohan K.B.,M.Tech.,Ph.D.,

Department of ECE,

ASIT, GUDUR-NELLORE(DT).

Submitted for viva – voce Examination held on :

Internal Examiner

External Examiner

AUDISANKARA INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

Accredited by NAAC with 'A+' Grade Approved by AICTE | Affiliated to JNTUA

NH5 Bypass Road, Gudur – 524101, Nellore (DT.), Andhra Pradesh



DECLARATION

We, **N.VMAHITHA (232H5A0413)**, **T.SWATHI (222H1A04K4)**, **V.SANTHOSH(222H1A04M2)**, **SK.LAHUR (222H1A04I5)** here by declare that the Project Work entitled– **IOT BASED AUTOMATIC VEHICLE ACCIDENT DETECTION AND RESCUE SYSTEM** done by us under the esteemed Guidance of **Dr P Sree Lakshmi , M.Tech,Ph.D Associate Professor, Dept. of ECE**. The Capstone Project report is submitted in partial fulfillment of the requirements for the award of the bachelor's degree in Electronics & Communication Engineering.

Date:

Place:

PROJECT ASSOCIATES

N.V.MAHITHA	232H5A0413
T.SWATHI	222H1A04K4
V.SANTHOSH	222H1A04M2
SK.LAHUR	222H1A04I5

ACKNOWLEDGEMENT

We express our deepest gratitude to **Dr. Vanki Penchalaiah**, Chairman of Audisankara College of Engineering & Technology, for providing us with an excellent academic environment and unwavering support throughout our project.

We extend our heartfelt thanks to **Smt. Vanki Aravinda**, Vice Chairperson, for her encouragement and continuous motivation, which have been instrumental in our academic journey.

We sincerely appreciate the guidance and support of **Dr. Amarendra , M.Tech , Ph.D.**, Principal whose leadership and vision have inspired us to strive for excellence.

Our sincere gratitude goes to **Dr. Kishore Mohan K.B, M.Tech., Ph.D.**, Head of the Department, for valuable insights, motivation, and support, which greatly contributed to the successful completion of our project.

We extend our heartfelt thanks to **Dr. P. Sree Lakshmi, M.Tech, Ph.D, Associate Professor, Dept. Of ECE** Project guide, for his/her invaluable insights, encouragement, and unwavering support, all of which played a significant role in the successful completion of our project.

We also extend our appreciation to **Dr. P. Sree Lakshmi, M.Tech. Ph.D**, Department Project Coordinator, for providing valuable feedback and ensuring the smooth progress of our project. We express our gratitude to our **department faculty** for their constant encouragement, technical guidance, and mentorship throughout this journey.

Our special thanks to the **non-teaching faculty** for their assistance and support in various aspects of our project work.

We would like to extend our heartfelt appreciation to our **parents and friends**, whose encouragement, patience, and moral support have been our driving force in completing this capstone project successfully.

CONTENTS

LIST OF FIGURES	i
LIST OF TABLES	ii
ABSTRACT	iii
CHAPTER 1: INTRODUCTION	1-2
1.1 Introduction	
CHAPTER 2: EMBEDDED SYSTEM	3-9
2.1 Embedded System	3-4
2.2 Categories of Embedded System	5-7
2.3 CPU Platforms	8
2.4 Peripherals	8
2.5 Tools	9
CHAPTER 3: METHODOLOGY	10-11
3.1 Existing System	10
3.2 Proposed System	10
3.3 System Architecture	11
CHAPTER 4: HARDWARE DESCRIPTION	12-36
4.1 Arduino Uno Board	12-19
4.2 LCD	20-23
4.3 Power supply	21-36

CHAPTER 5: SOFTWARE DESCRIPTION	37-51
5.1 Arduino IDE	37-43
5.2 Libraries	44-46
5.3 flow of Software Execution	47-51
CHAPTER 6:CONCLUSION	52
CHAPTER 7:REFERENCES	53

LIST OF FIGURES

Fig 1.1 Block Diagram of IoT Based Automatic Vehicle Detection and Rescue System

Fig 2.1 Basic Structure of an Embedded System

Fig 2.2 Categories of Embedded Systems

Fig 2.3 CPU Platform Architecture

Fig 2.4 Embedded System Peripherals

Fig 3.1 Existing Vehicle Accident Detection System

Fig 3.2 Proposed System Architecture

Fig 3.3 System Flowchart

Fig 4.1 Arduino Uno Board

Fig 4.2 LCD (Liquid Crystal Display)

Fig 4.3 GPS Module

Fig 4.4 GSM Module

Fig 4.5 Vibration/Accident Detection Sensor

Fig 4.6 Complete Hardware Setup

Fig 5.1 Arduino IDE Interface

Fig 5.2 Program Flow Diagram

Fig 5.3 IoT Cloud Monitoring Interface

Fig 5.4 SMS Alert Output Screenshot

LIST OF TABLES

Table 2.1 Comparison of Embedded System Categories

Table 3.1 Comparison Between Existing and Proposed System

Table 4.1 Arduino Uno Technical Specifications

Table 4.2 GPS Module Specifications

Table 4.3 GSM Module Specifications

Table 4.4 LCD Specifications

Table 4.5 Sensor Specifications

Table 5.1 Software Requirements

Table 5.2 Pin Configuration of Arduino

Table 5.3 Testing Results of the System

Table 5.4 Alert Message Format

ABSTRACT

IOT BASED AUTOMATIC VEHICLE ACCIDENT DETECTION AND RESCUE SYSTEM

The rapid rise of technology and infrastructure has made our lives easier. The high demand of automobiles has also increased the traffic hazards and road accidents. Life of the people is under high risk. The delay in reaching of the ambulance to the accident location and the traffic congestion in between accident location and hospital increases the chances of death of the victim. To overcome this problem our automatic ambulance rescue system comes to the rescue. This proposed IOT based accident detection system helps to reduce the loss of life due to accidents and also reduces the time taken by the ambulance to reach the hospital. To detect the accident there is vibration sensor present in this rescue system and the WIFI module included sends messages about the location to the respective guardian and rescue team. The rapid growth of technology and transportation has significantly increased the number of vehicles on the road, leading to a rise in traffic congestion and road accidents. Delay in providing immediate medical assistance after an accident is one of the major reasons for increased fatalities. Timely detection and quick response play a crucial role in saving lives. To address this issue, this project proposes an IoT-based Automatic Vehicle Accident Detection and Rescue System that ensures faster emergency response and reduces loss of life. The proposed system uses sensors such as vibration sensors and impact detection modules to identify accidents in real time. When a collision or abnormal vibration beyond a threshold limit is detected, the system automatically activates. A GPS module determines the exact location of the accident site, and a GSM or Wi-Fi module transmits the location details to predefined emergency contacts, nearby hospitals, rescue teams, and guardians. This reduces the delay caused by manual reporting and ensures that medical assistance reaches the victim as quickly possible.

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION:

The development in the field of automobiles is highly increasing and which leads to the accidents and so many hazards due to traffic. People's life are under high risk. This situation prevails, just because there is a lack of emergency facilities in our country. In our country, many people loose their life with accidents. Because of causalities or improper communication to rescue team. We are in the process of solving this issue by proposing an efficient solution and to reduce the loss of lives as much as possible. In our theory, the design of the system help us to detect accidents in significantly less time and transfer the fundamental information to the first aid centre within a few seconds covering the geographical coordinates, the time and the angle where the vehicle had met with an accident. This alert message is sent to the rescue team(ambulance) and the family within the short period. This real time application saves many valuable lives .The message is sent through the GSM module and the location of the The basic idea is to localize the vehicle system by receiving the real time position of the vehicle through GPS and send the information through GSM module via SMS service with an added feature of GPRS transmission to the monitoring center through usage of internet [M.AL-Rousan, A. R. AI-Ali and K. Darwish et al, 2004]. Using AT89S52, this project has been designed. It used EEPROM to store the phone numbers. accident is detected with the help of the GPS module. The accident can be detected precisely with the help of both Micro electro mechanical system (MEMS) sensor and vibration sensor. The Angle of the rolls over of the car can also be known by the message through the MEMS sensor. This application provides the optimum solution to poor emergency facilities provided to the roads accidents in the most feasible way.

At present criteria, we cannot detect where the accident has occurred and hence no information related to it, leading to the death of an individual. The research work is going on for tracking of the vehicle even in dark clumsy areas where there is no network for receiving the signals. In literature, a number of approaches to provide security and safety through monitoring the vehicle's real time precise positioning and information using different tech- nologies have been proposed.

A good survey of using GPS, GSM and GIS has been provided in [IoanLita, Ion BogdanCioc, Daniel AlexandruVisan et al, 2006] and [Mrs. RamyaKulandaivel, P.Ponmalar, B.Geetha, G.Saranya et al, 2012]. The general mechanism is to provide the real time geographical position of a vehicle using GPS receiver and send this information to GSM center through configurable software, this is all done by the monitoring center which is working as a control unit that is connected not only by an optical cable but also connected wirelessly through TCP/IP protocols. The monitoring center distributes the data to the client in an understandable format and it also stores the travelling records and displays the real time information about vehicle on electronic map through GIS system [IoanLita, Ion BogdanCioc, Daniel AlexandruVisan et al, 2006]. Another approach is that vehicle terminal includes a GPS receiver which extracts information about position through GPS satellites and sends it through GSM network and to the control center which reads.

CHAPTER 2

EMBEDDED SYSTEM

2.1 EMBEDDED SYSTEM :

An Embedded System is a combination of computer hardware and software, and perhaps additional mechanical or other parts, designed to perform a specific function. An embedded system is a microcontroller-based, software driven, reliable, real-time control system, autonomous, or human or network interactive, operating on diverse physical variables and in diverse environments and sold into a competitive and cost-conscious market.

An embedded system is not a computer system that is used primarily for processing, not a software system on PC or UNIX, not a traditional business or scientific application. High-end embedded & lower end embedded systems. High-end embedded system - Generally 32, 64 Bit Controllers used with OS. Examples Personal Digital Assistant and Mobile phones etc. .Lower end embedded systems - Generally 8,16 Bit Controllers used with a minimal operating systems and hardware layout designed for the specific purpose.

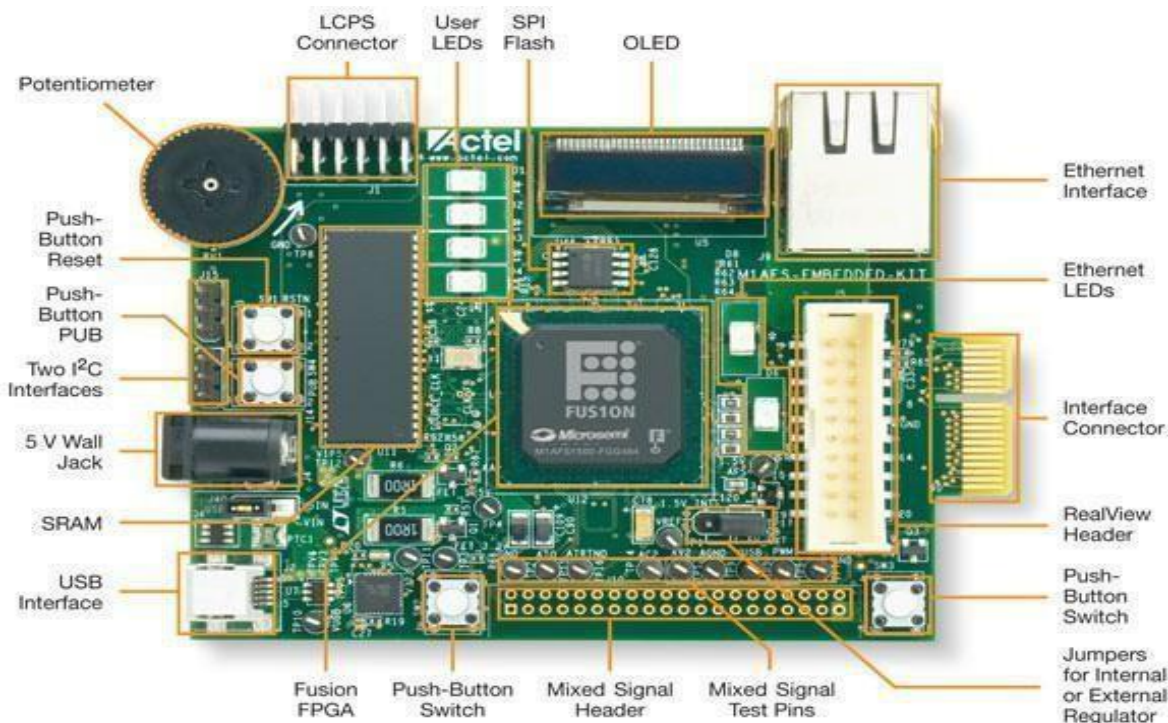


Figure 1.1 Basic Embedded System

2.1.2 Eembedded System Design Cycle:

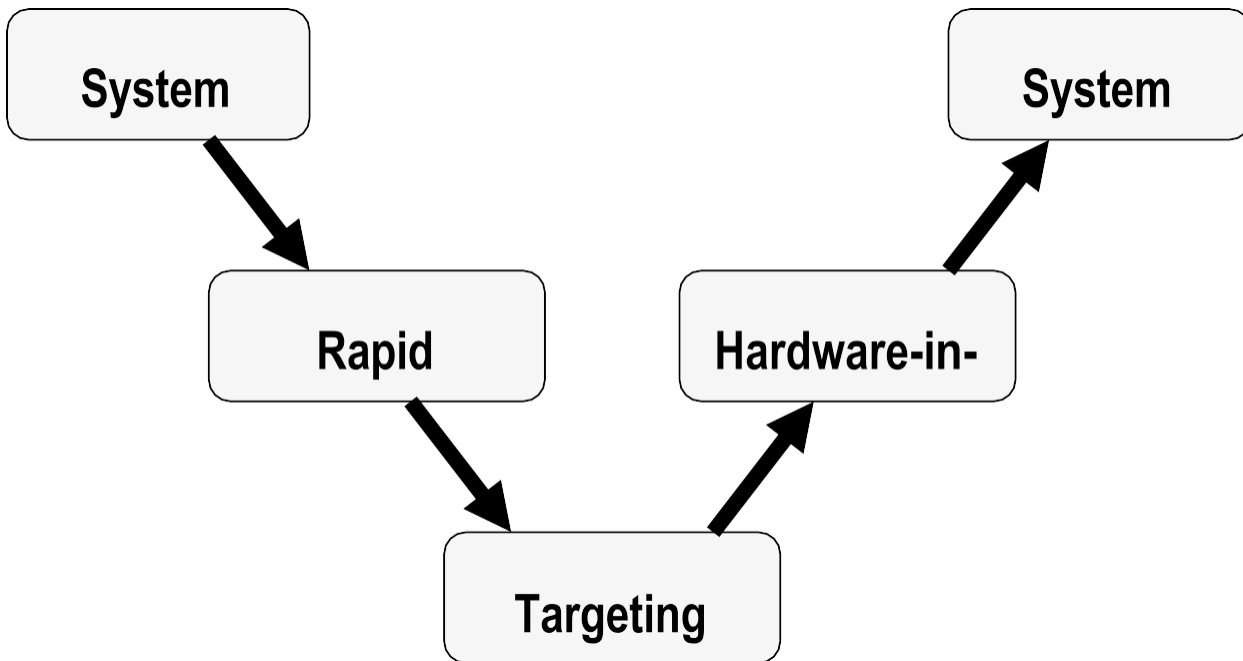


Fig1.2. V Diagram

2.1.3. Characteristics of Embedded System:

An embedded system is any computer system hidden inside a product other than a computer. They will encounter a number of difficulties when writing embedded system software in addition to those we encounter when we write applications. Throughput – Our system may need to handle a lot of data in a short period of time. Response–Our system may need to react to events quickly. Testability–Setting up equipment to test embedded software can be difficult. Debugability–Without a screen or a keyboard, finding out what the software is doing wrong (other than not working) is a troublesome problem. Reliability – embedded systems must be able to handle any situation without human intervention.

Memory space – Memory is limited on embedded systems, and you must make the software and the data fit into whatever memory exists. Program installation – you will need special tools to get your software into embedded systems. Power consumption – Portable systems must run on battery power, and the software in these systems must conserve power. Processor hogs – computing that requires large amounts of CPU time can complicate the response problem. Cost – Reducing the cost of the hardware is a concern in many embedded system projects;

software often operates on hardware that is barely adequate for the job. Embedded systems have a microprocessor/ microcontroller and a memory. Some have a serial port or a network connection. They usually do not have keyboards, screens or disk drives.

2.1.4 Applications:

- Military and aerospace embedded software applications
- Communication Applications.
- Industrial automation and process control software.
- Mastering the complexity of applications.
- Reduction of product design time.

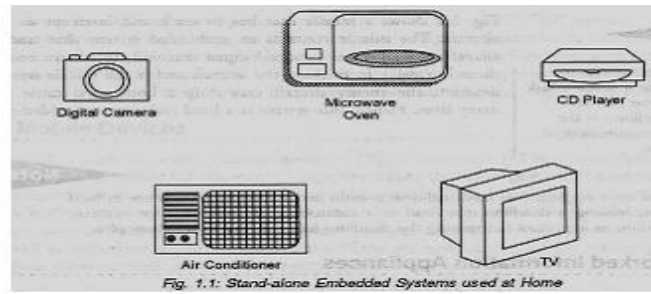
2.2 Categories of Embedded Systems:

Based on functionality and performance requirements, embedded systems can be categorized as:

- Stand-alone embedded systems
- Real-time systems
- Networked information appliances
- Mobile devices

2.2.1 Stand-alone Embedded Systems:

As the name implies, stand-alone systems work in stand-alone mode. They take inputs, process them and produce the desired output. The input can be electrical signals from transducers or commands from a human being such as the pressing of a button. The output can be electrical signals to drive another system, an LED display or LCD display for displaying of information to the users. Embedded systems used in process control, automobiles, consumer electronic items etc. fall into this category. In a process control system, the inputs are from sensors that convert a physical entity such as temperature or pressure into its equivalent electrical signal. A few embedded systems used at home are shown in figure.



5

Figure1.3: Standard Embeddes Systems used at Home

2.2.2 Real Time Systems:

Embedded systems in which some specific work has to be done in a specific time period are called real-time systems. For example: consider a system that has to open a valve within 30milliseconds when the humidity crosses a particular threshold. If the valve is not opened within 30 milliseconds, a catastrophe may occur. Such systems with strict deadlines are called hard real-time systems. In some embedded systems, deadlines are imposed but not a adhering to them once in a while may not lead to a catastrophe. For example, consider a DVD player. Suppose, you give a command to the DVD player from are mote control, and there is a delay of a few milliseconds in executing that command. But this delay won't lead to a serious simplification. Such systems are called soft real-time systems.

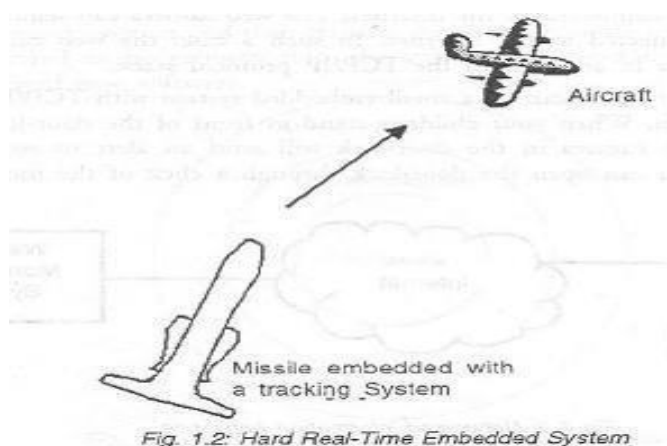


Fig 1.4.: Hard Real-Time Embedded System

2.2.3 Networked Information Appliances:

Embedded systems that are provided with network interfaces and accessed by networks such as Local Area Network or the Internet are called networked information appliances. Such embedded systems are connected to a network, typically a network running TCP/IP (Transmission Control Protocol! Internet Protocol) protocol suite, such as the Internet or a company's Intranet. These systems have emerged in recent years. These systems run the protocol TCP/IP stack and get connected either through PPP or Ethernet to a network and communicate with other nodes in the network. Here are some examples of such systems:

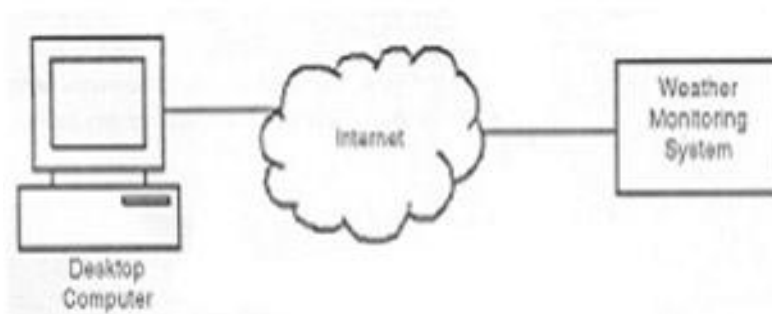


Fig. 1.3: Networked Information Appliance

Fig 1.5.: Networked Information Appliance

Mobile Devices:

Mobile devices such as mobile phones, Personal Digital Assistants, smart phones etc. are a special category of embedded systems. Though the PDAs do many general purpose tasks, they need to be designed just like the 'conventional' embedded systems. The limitations of –the mobile devices- memory constraints, small size, lack of good user interfaces such as full-fledged keyboard and display etc.-are same as those found in the embedded systems discussed above. Hence, mobile devices are considered as embedded systems. However, the PDAs are now capable of supporting general-purpose application software such as word processors, games, etc.

2.2.3 User Interfaces:

Embedded systems range from no user interface at all - dedicated only to one task - to full user interfaces similar to desktop operating systems in devices such as PDAs.

2.2.4 Simple Systems:

Simple embedded devices use buttons, LEDs, and small character- or digit-only displays, often with a simple menu system.

2.2.5 In More Complex Systems:

A full graphical screen, with touch sensing or screen-edge buttons provides flexibility while minimizing space is used. The meaning of the buttons can change with the screen, and selection involves the natural behavior of pointing at what's desired. Handheld systems often have a screen with a "joystick button" for a pointing device.

2.3 CPU Platform:

Embedded processors can be broken into two distinct categories: microprocessors and micro controllers. Micro controllers have built-in peripherals on the chip, reducing size of the system. There are many different CPU architectures used in embedded designs such as ARM, MIPS, Cold fire/68k, PowerPC, x86, PIC, 8051, Atmel AVR, Renesas H8, SH, V850, FR-V, M32R, Z80, Z8, etc. A common configuration for very-high-volume embedded systems is the system on a chip an application-specific integrated circuit for which the CPU core was purchased and added as part of the chip design.

2.4 Peripherals:

Embedded Systems talk with the outside world via peripherals, such as:

- Serial Communication Interfaces
- Synchronous Serial Communication Interface
- Universal Serial Bus
- Networks: Controller Area Network, Lon Works, etc
- Timers: Capture/Compare and Time Processing Units
- Discrete IO: General Purpose Input Output

2.5 Tools:

As for other software, embedded system designers use compilers, assemblers, and debuggers to develop embedded system software. However, they may also use some more specific tools:

- An in-circuit emulator is a hardware device that replaces or plugs into the microprocessor, and provides facilities to quickly load and debug experimental code in the system.
- Utilities to add a checksum to a program, so the embedded system can check if the program is valid.
- For systems using digital signal processing, developers may use a math workbench such as Mathematic to simulate the mathematics.
- An embedded system may have its own special language or design tool.

CHAPTER-3

METHODOLOGY

3.1. EXISTING SYSTEM:

Manual Detection System :

In this method, accident is detected from

- motorist report
- transportation department
- public crews report
- aerial surveillance
- close circuit camera surveillance

The drawback of this type of detection system is that someone has to witness the incident.

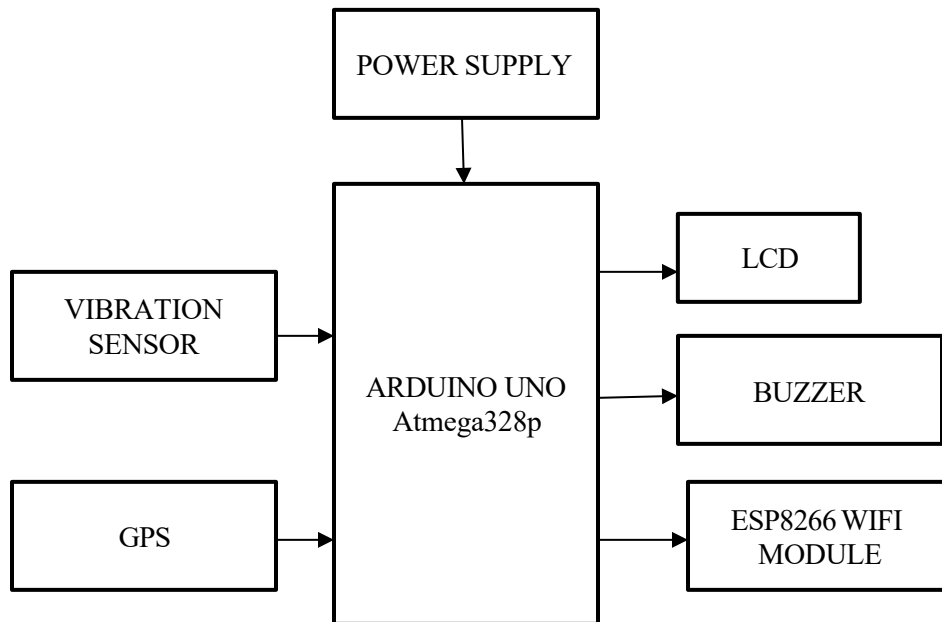
Driver initiated detection system. Moreover, there are delays and inaccuracies due to the expression problem of the witness.

3.2. PROPOSED SYSTEM:

This proposed IOT based accident detection system helps to reduce the loss of life due to accidents and also reduces the time taken by the ambulance to reach the hospital. To detect the accident there is vibration sensor present in this rescue system and the WIFI module included sends messages about the location to the respective guardian and rescue team.

With the help of vibration sensor signal, a severe accident due to an obstacle can be recognized. Microcontroller used, sends the alert message through the WIFI module including the location to guardian or a rescue team. So, the emergency help team can immediately trace the location through the GPS module, after receiving the accident location information, action can be taken immediately. This vibration-based accident detection system is powered by At mega 328 microcontroller it consists of display, vibration sensor, WIFI module and alarm. This automatic ambulance rescue system project is useful in detecting the accident.

BLOCK DIAGRAM:



CHAPTER-4

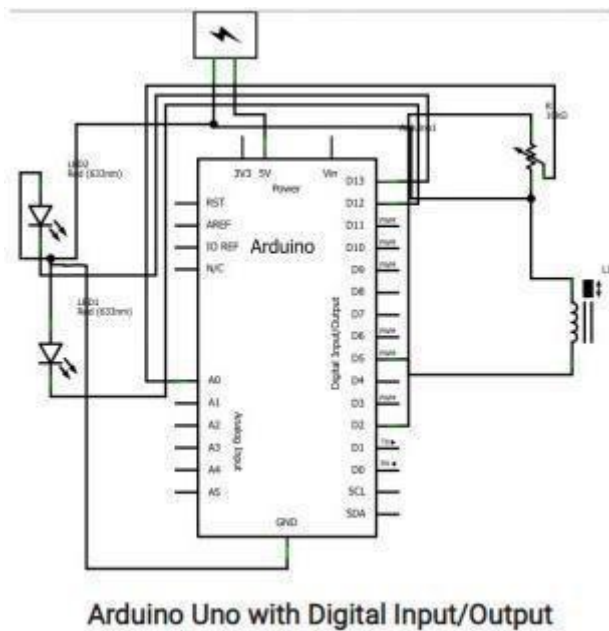
HARDWARE DESCRIPTION

4.1. Arduino Uno Board:

Arduino is a single-board microcontroller meant to make the application more accessible which are interactive objects and its surroundings. The hardware features with an open-source hardware board designed around an 8-bit Atmel AVR microcontroller or a 32-bit Atmel ARM. Current models consists a USB interface, 6 analog input pins and 14 digital I/O pins that allows the user to attach various extension boards.

The Arduino Uno board is a microcontroller based on the ATmega328. It has 14 digital input/output pins in which 6 can be used as PWM outputs, a 16 MHz ceramic resonator, an ICSP header, a USB connection, 6 analog inputs, a power jack and a reset button. This contains all the required support needed for microcontroller. In order to get started, they are simply connected to a computer with a USB cable or with a AC-to-DC adapter or battery. Arduino Uno Board varies from all other boards and they will not use the FTDI USB-to-serial driver chip in them. It is featured by the Atmega16U2 (Atmega8U2 up to version R2) programmed as a USB-to-serial converter.





There are various types of Arduino boards in which many of them were third-party compatible versions. The most official versions available are the Arduino Uno R3 and the Arduino Nano V3. Both of these run a 16MHz Atmel ATmega328P 8-bit microcontroller with 32KB of flash RAM 14 digital I/O and six analogue I/O and the 32KB will not sound like as if running Windows. Arduino projects can be stand-alone or they can communicate with software on running on a computer. For e.g. Flash, Processing, Max/MSP). The board is clocked by a 16 MHz ceramic resonator and has a USB connection for power and communication. You can easily add micro-SD/SD card storage for bigger tasks.

Features of the Arduino Uno Board:

- It is an easy USB interface. This allows interface with USB as this is like a serial device.
- The chip on the board plugs straight into your USB port and supports on your computer as a virtual serial port. The benefit of this setup is that serial communication is an extremely easy protocol which is time-tested and USB makes connection with modern computers and makes it comfortable.
- It is easy-to-find the microcontroller brain which is the ATmega328 chip. It has more number of hardware features like timers, external and internal interrupts, PWM pins and multiple sleep modes.

- It is an open-source design and there is an advantage of being open source is that it has a large community of people using and troubleshooting it. This makes it easy to help in debugging projects.
- It is a 16 MHz clock which is fast enough for most applications and does not speed up the microcontroller.
- It is very convenient to manage power inside it and it had a feature of built-in voltage regulation. This can also be powered directly off a USB port without any external power. You can connect an external power source of up to 12v and this regulates it to both 5v and 3.3v.
- 13 digital pins and 6 analog pins. This sort of pins allows you to connect hardware to your Arduino Uno board externally. These pins are used as a key for extending the computing capability of the Arduino Uno into the real world. Simply plug your electronic devices and sensors into the sockets that correspond to each of these pins and you are good to go.
- This has an ICSP connector for bypassing the USB port and interfacing the Arduino directly as a serial device. This port is necessary to re-boatload your chip if it corrupts and can no longer be used to your computer.
- It has a 32 KB of flash memory for storing your code.
- An on-board LED is attached to digital pin 13 to make fast the debugging of code and to make the debug process easy.
- Finally, it has a button to reset the program on the chip.

Arduino was created in the year 2005 by two Italian engineers David Cuartielles and Massimo Banzi with the goal of keeping in mind about students to make them learn how to program the Arduino uno microcontroller and improve their skills about electronics and use it in the real world.

Arduino uno microcontroller can sense the environment by receiving input from a variety of sensors and can affect its surroundings by controlling lights, motors, and other actuators. The microcontroller is programmed using the Arduino programming language (based on Wiring) and the Arduino development environment (based on Processing).

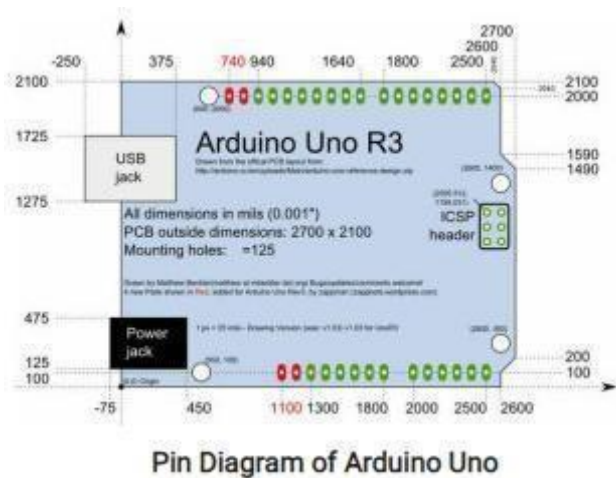
ATmega168/328-Arduino Pin Mapping:



ATmega168-328Arduino Pin Mapping

Programming:

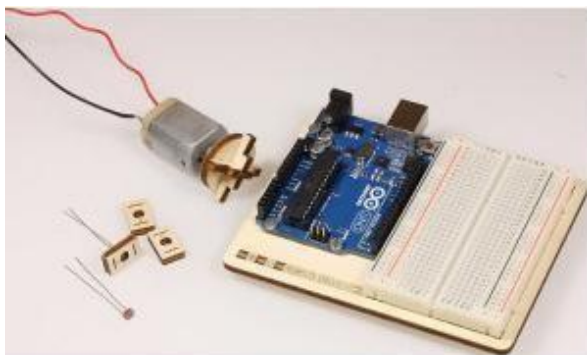
- The Arduino integrated development environment (IDE) is a cross-platform application written in Java, and is derived from the IDE for the Processing programming language and the Wiring projects
- The Arduino Uno board can be programmed with the Arduino software.
- Select “Arduino Uno from the Tools > Board menu (according to the microcontroller on your board).
- The ATmega328 on the Arduino Uno comes returned with a bootloader that allows you to upload new code to it without the use of an external hardware programmer. It communicates using the original STK500 protocol.
- You can also bypass the bootloader and program the microcontroller through the ICSP (In-Circuit Serial Programming) header.
- The ATmega16U2 (or 8U2 in the rev1 and rev2 boards) firmware source code is available .



The ATmega16U2/8U2 is loaded with a DFU bootloader, which can be activated by:

- On Rev1 boards: connecting the solder jumper on the back of the board (near the map of Italy) and then resetting the 8U2.
- On Rev2 or later boards: there is a resistor that pulling the 8U2/16U2 HWB line to ground, making it easier to put into DFU mode.

You can then use Atmel's FLIP software (Windows) or the DFU programmer (Mac OS X and Linux) to load a new firmware. Or you can use the ISP header with an external programmer (overwriting the DFU bootloader).



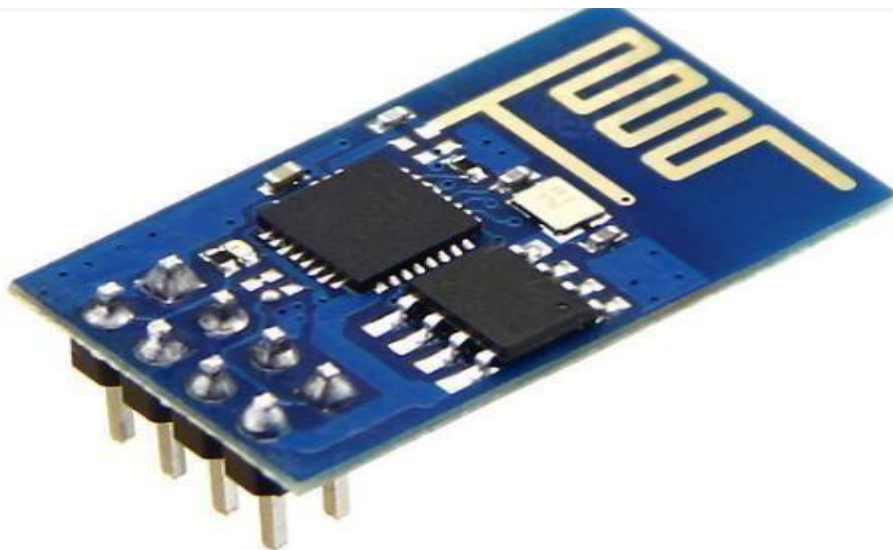
Arduino Uno Starter Kit

Microcontroller	ATmega328
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
Analog Input Pins	6
DC Current per I/O Pin	40 Ma
DC Current for 3.3V Pin	50 Ma
Flash Memory	32 KB (ATmega328) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328)
EEPROM	1 KB (ATmega328)
Clock Speed	16 MHz

ESP8266 WIFI Module

Introduction

ESP8266 is Wi-Fi enabled system on chip (SoC) module developed by Espressif system. It is mostly used for development of IoT (Internet of Things) embedded applications.



ESP8266-01 WiFi Module

ESP8266 comes with capabilities of

- 2.4 GHz Wi-Fi (802.11 b/g/n, supporting WPA/WPA2),
- general-purpose input/output (16 GPIO),
- Inter-Integrated Circuit (I²C) serial communication protocol,
- analog-to-digital conversion (10-bit ADC)
- Serial Peripheral Interface (SPI) serial communication protocol,
- I²S (Inter-IC Sound) interfaces with DMA(Direct Memory Access) (sharing pins with GPIO),
- UART (on dedicated pins, plus a transmit-only UART can be enabled on GPIO2), and
- pulse-width modulation (PWM).

It employs a 32-bit RISC CPU based on the Tensilica Xtensa L106 running at 80 MHz (or overclocked to 160 MHz). It has a 64 KB boot ROM, 64 KB instruction RAM and 96 KB data RAM. External flash memory can be accessed through SPI.

ESP8266 module is low cost standalone wireless transceiver that can be used for end-point IoT developments.

To communicate with the ESP8266 module, microcontroller needs to use set of AT commands. Microcontroller communicates with ESP8266-01 module using UART having specified Baud rate.

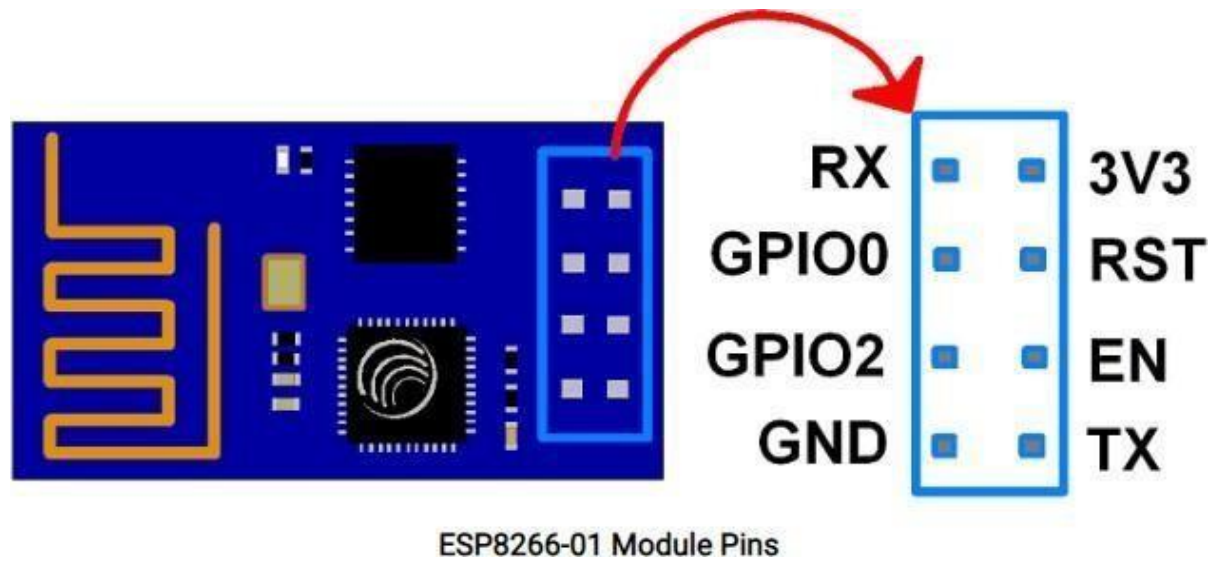
There are many third-party manufacturers that produce different modules based on this chip. So, the module comes with different pin availability options like,

- ESP-01 comes with 8 pins (2 GPIO pins) – PCB trace antenna. (shown in above figure)
- ESP-02 comes with 8 pins, (3 GPIO pins) – U-FL antenna connector.
- ESP-03 comes with 14 pins, (7 GPIO pins) – Ceramic antenna.
- ESP-04 comes with 14 pins, (7 GPIO pins) – No ant.

etc.

For example, below figure shows ESP-01 module pins

ESP8266-01 Module Pin Description



3V3: - 3.3 V Power Pin.

GND: - Ground Pin.

RST: - Active Low Reset Pin.

EN: - Active High Enable Pin.

TX: - Serial Transmit Pin of UART.

RX: - Serial Receive Pin of UART.

GPIO0 & GPIO2: - General Purpose I/O Pins. These pins decide what mode (boot or normal) the module starts up in. It also decides whether the TX/RX pins are used for Programming the module or for serial I/O purpose.

To program the module using UART, Connect GPIO0 to ground and GPIO2 to VCC or leave it open. To use UART for normal Serial I/O leave both the pins open (neither VCC nor Ground).

4.2. LCD (Liquid Cristal Display) :

A liquid crystal display (LCD) is a thin, flat display device made up of any number of color or monochrome pixels arrayed in front of a light source or reflector. Each pixel consists of a column of liquid crystal molecules suspended between two transparent electrodes, and two polarizing filters, the axes of polarity of which are perpendicular to each other. Without the liquid crystals between them, light passing through one would be blocked by the other. The liquid crystal twists the polarization of light entering one filter to allow it to pass through the other.

A program must interact with the outside world using input and output devices that communicate directly with a human being. One of the most common devices attached to an controller is an LCD display. Some of the most common LCDs connected to the controllers are 16X1, 16x2 and 20x2 displays. This means 16 characters per line by 1 line 16 characters per line by 2 lines and 20 characters per line by 2 lines, respectively.

Data can be placed at any location on the LCD.

For 16×1 LCD, the address locations are:

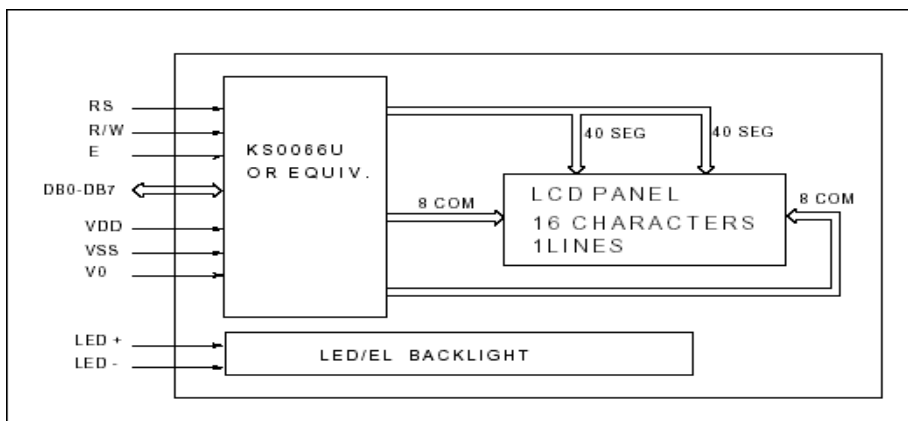
POSITION		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
ADDRESS	LINE1	00	01	02	03	04	05	06	07	40	41	42	43	44	45	46	47

Table 5 : Address locations for a 1x16 line LCD

Even limited to character based modules there is still a wide variety of shapes and sizes available. Line lengths of 8,16,20,24,32 and 40 characters are all standard, in one, two and four line versions. Several different LC technologies exists. “supertwist” types, for example, offer Improved contrast and viewing angle over the older “twisted nematic” types.

Some modules are available with back lighting, so so that they can be viewed in dimly-lit conditions. The back lighting may be either “electro-luminescent”, requiring a high voltage inverter circuit, or simple LED illumination.

Electrical Block Diagram:



PIN DESCRIPTION:

Most LCDs with 1 controller has 14 Pins and LCDs with 2 controller has 16 Pins (two pins are extra in both for back-light LED connections).

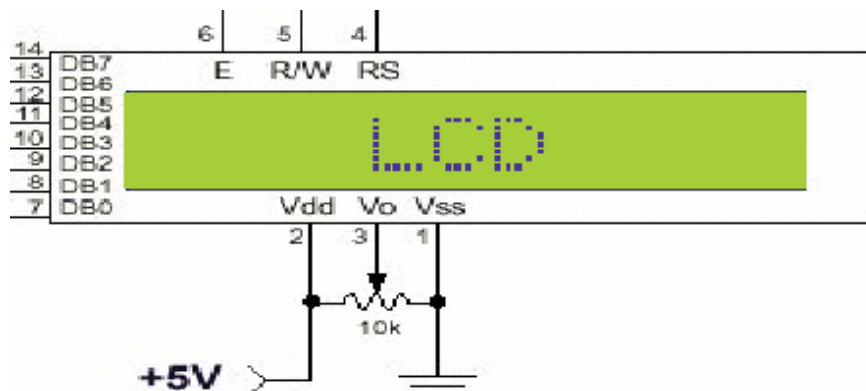


Table 6 : Table for LCD

PIN	SYMBOL	FUNCTION
1	Vss	Power Supply(GND)
2	Vdd	Power Supply(+5V)
3	Vo	Contrast Adjust
4	RS	Instruction/Data Register Select
5	R/W	Data Bus Line
6	E	Enable Signal
7-14	DB0-DB7	Data Bus Line
15	A	Power Supply for LED B/L(+)
16	K	Power Supply for LED B/L(-)

CONTROL LINES:

EN: Line is called "Enable." This control line is used to tell the LCD that you are sending it data. To send data to the LCD, your program should make sure this line is low (0) and then set the other two control lines and/or put data on the data bus. When the other lines are completely ready, bring EN high (1) and wait for the minimum amount of time required by the LCD datasheet (this varies from LCD to LCD), and end by bringing it low (0) again.

RS: Line is the "Register Select" line. When RS is low (0), the data is to be treated as a command or special instruction (such as clear screen, position cursor, etc.). When RS is high (1), the data being sent is text data which could be displayed on the screen. For example, to display the letter "T" on the screen you would set RS high.

RW: Line is the "Read/Write" control line. When RW is low (0), the information on the data bus is being written to the LCD. When RW is high (1), the program is effectively querying (or reading) the LCD. Only one instruction ("Get LCD status") is a read command. All others are write commands, so RW will almost always be low.

Logic status on control lines:

- E - 0 Access to LCD disabled
- 1 Access to LCD enabled
- R/W - 0 Writing data to LCD
- 1 Reading data from LCD
- RS - 0 Instructions
- 1 Character

Writing data to the LCD:

- Set R/W bit to low
- Set RS bit to logic 0 or 1 (instruction or character)
- Set data to data lines (if it is writing)
- Set E line to high
- Set E line to low

Read data from data lines (if it is reading)on LCD:

- Set R/W bit to high
- Set RS bit to logic 0 or 1 (instruction or character)
- Set data to data lines (if it is writing)
- Set E line to high
- Set E line to low

Entering Text:

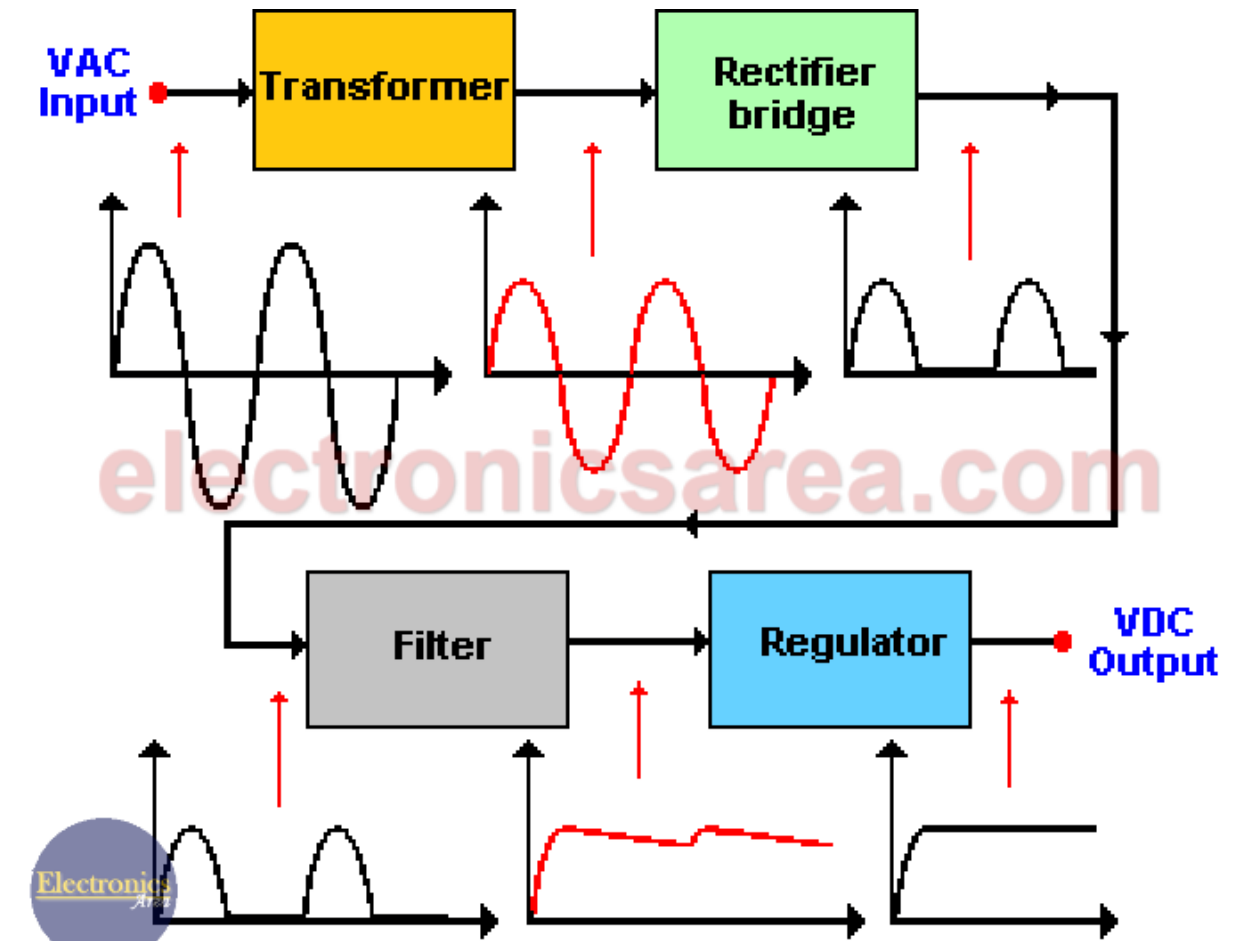
First, a little tip: it is manually a lot easier to enter characters and commands in hexadecimal rather than binary (although, of course, you will need to translate commands from binary couple of sub-miniature hexadecimal rotary switches is a simple matter, although a little bit into hex so that you know which bits you are setting).

POWER SUPPLY:

Power Supply block Diagram

Many electronic circuits need a direct current (DC) voltage source, but what we commonly find are voltage sources of alternating current (AC). In order to achieve a direct current voltage source, the alternating current input must follow a conversion process like the one shown in the **power supply block diagram** below

The image shows the main components of a basic power supply diagram and the waveforms at the beginning (AC input), at the end (DC output) and between blocks.



Input signal which goes to the transformer primary winding is a sine wave and its amplitude depends on the country's electric distribution system (110/220 VAC or other). See the basic units of measurement in electronics.

Power Supply block diagram parts:

Electrical Transformer:

The electrical transformer receives on the primary winding an AC voltage and delivers on the secondary winding a different AC voltage (a lower one). This AC output voltage must be according to the DC voltage we want to obtain at the end.

For example: If we want a 12 VDC output, the secondary winding of the transformer must have an AC voltage no less of 9 volts.



Electric transformer

The peak value at the transformer secondary winding is $V_p = 1.41 \times 9 = 12.69$ volts. Even though this value is very close to the one we wanted to get, it is not recommended because we need to take into account the voltage drops at different stages (blocks) of the power supply. In this case, we can choose a transformer with a 12 volts AC secondary winding. With this AC voltage, we can get a peak voltage of: $V_p = 12 \times 1.41 = 16.92$ volts. Note: $V_{peak} = V_{rms} \times 1.41$

Rectifier bridge (rectifier diodes):

The rectifier bridge transforms the secondary winding AC voltage into a pulsating DC voltage. (look at the diagram). In our case, we use a $\frac{1}{2}$ wave rectifier, then we eliminate the negative part of the wave.



Rectifier diode

Filter (capacitors):

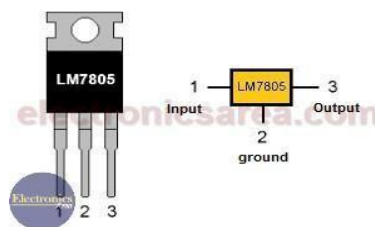
Filter are one or more electrolytic capacitors in parallel that flattens or smooths the previous wave eliminating the alternating current (AC) component delivered by the rectifier. These capacitors are charged to the maximum voltage value that the rectifier can deliver, and they are discharged when the pulsating signal disappears. Look at the picture above.



Electrolytic Capacitor

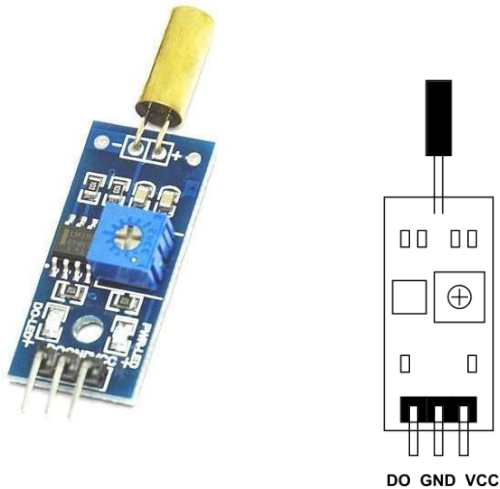
Voltage regulator:

The voltage regulator receives the signal from the filter and delivers a constant voltage (let's say 12 DC volts) regardless of the variations on the load or the voltage supply. Voltage regulator can be implemented in several ways. It can be a transistorized voltage regulator or a monolithic voltage regulator. The image below shows the LM7805 voltage regulator (5VDC output). You can also find the LM7812 voltage regulator (12VDC output).



LM7805 Voltage regulator

SW-520D - Tilt Sensor Module



SW-520D Tilt Sensor SW-520D Tilt Sensor Pinout

TILT SENSOR MODULE is a device used for knowing the planar movement. Although they are available in various types their basic function remains the same. Their function is to detect the plane shift from horizontal to vertical and sent of a signal when it happens. There are modules which could sense even small plane shifts but here we are going to discuss about simple contact type TILT SENSOR module. Here we are going to use **SW-520D TILT SENSOR MODULE**.

TILT SENSOR MODULE Pin Configuration

TILT SENSOR MODULE is three pin devices as shown in figure. All three pins are compulsory for using the module.

Pin Name	Description
GND	Connected to GROUND
VCC	Connected to +5V
DO	Output of TILT SENSOR

TILT SENSOR MODULE Features and Specifications

- Supply voltage: 3.3 V to 5V
- Output can directly connect to controller
- TTL level output
- Maximum output current : 15mA
- Can work on low voltages
- Maximum operating temperature: 0°C to + 80°C
- Easy interface
- Long life.

Where TILT SENSOR MODULES are Used?

Here are few cases where **SW-520D SENSOR MODULE** is chosen:

Case1: SW-520D is used when the cost of system cannot be high. SW-520D is one of cheapest and robust TILT SENSORS on the market. It is easily available to replace in case of fault.

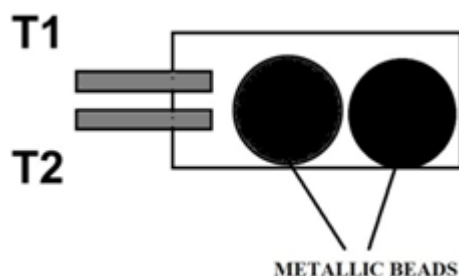
Case2: SW-520D is easy to install in the system. All you need to do is power the module appropriately and take the output correctly.

Case3: With very long life this module can be used on permanent installment.

Also the module does not need any additional components to install in a system.

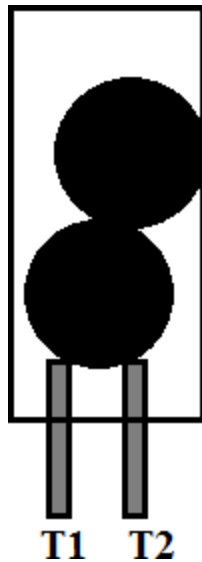
How to Use SW-520D TILT SENSOR MODULE

Before understanding how to use the module let us first understand the **working of SW-520D tilt switch**. For that consider the internal working of this tilt switch.

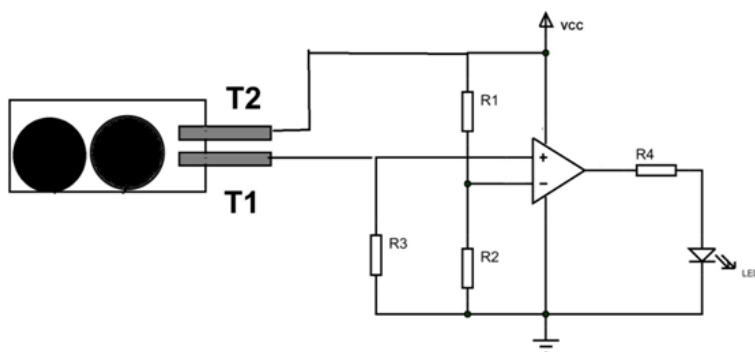


As shown in figure inside SW-520D tilt sensor all we have is two metallic beads. And the output terminals are projected to the inside. When the tilt switch lies horizontally on the plane, the two metallic balls rest on the floor as shown in figure. Under this situation there will no contact between terminals T1 and T2. So T1 and T2 will be open.

Now when the tilt sensor changed position from horizontal to vertical, we have something like this.



When the tilt switch is moved vertical because of gravity, the metallic balls come to rest on the projected terminals. Because the ball is metallic, when they rest on terminals an electrical contact is formed between two terminals. So, in this position T1 and T2 are short circuited. When the tilt switch is moved to horizontal position again, ball leaves the contact to rest on floor as before. With that T1 and T2 contact breaks making them open circuit. So, **SW-520D** output terminals open circuit when the body lies horizontal and is short circuit when body is vertical. Hence using this tilt switch we can detect which plane the body is lying. This tilt switch contact will have noise and cannot be connected to controllers directly. So, we will connect this **tilt switch to a simple op-amp comparator circuit** to make a sensor module.



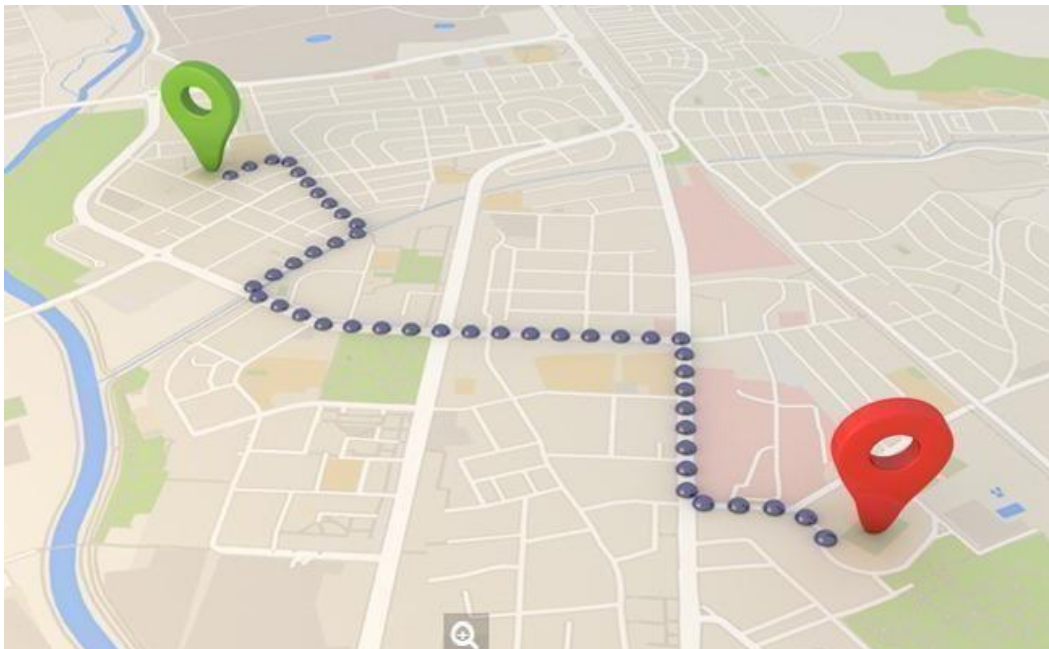
Simplified tilt sensor module is shown in figure. Under normal situations T1 and T2 will be open and the entire VCC appears across it. So, voltage at positive terminal of comparator will be zero. So, the output of comparator will be low. At this time LED will be OFF. When the tilt sensor is moved vertical, T1 and T2 will short circuit. At this time entire VCC appears across R3. At this time voltage at positive terminal of comparator will be VCC. With HIGH at positive terminal output of comparator will be HIGH. At this time LED will be ON. This module output can be given to controller or other systems for using appropriately.

Applications

- Security systems.
- Digital cameras
- Toys
- Motors
- Robotic arms
- Vending machines.
- Measuring instruments.
- Hobby projects.

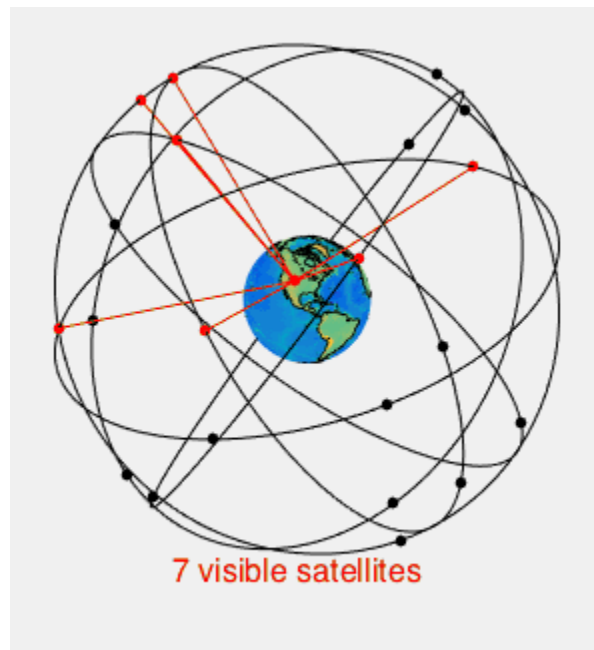
GPS Receiver Module

Introduction



GPS receivers are generally used in smartphones, fleet management system, military etc. for tracking or finding location. Global Positioning System (GPS) is a satellite-based system that uses satellites and ground stations to measure and compute its position on Earth. PS is also known as Navigation System with Time and Ranging (NAVSTAR) GPS. GPS receiver needs to receive data from at least 4 satellites for accuracy purpose. GPS receiver does not transmit any information to the satellites. This GPS receiver is used in many applications like smartphones, Cabs, Fleet management etc.

How GPS Works

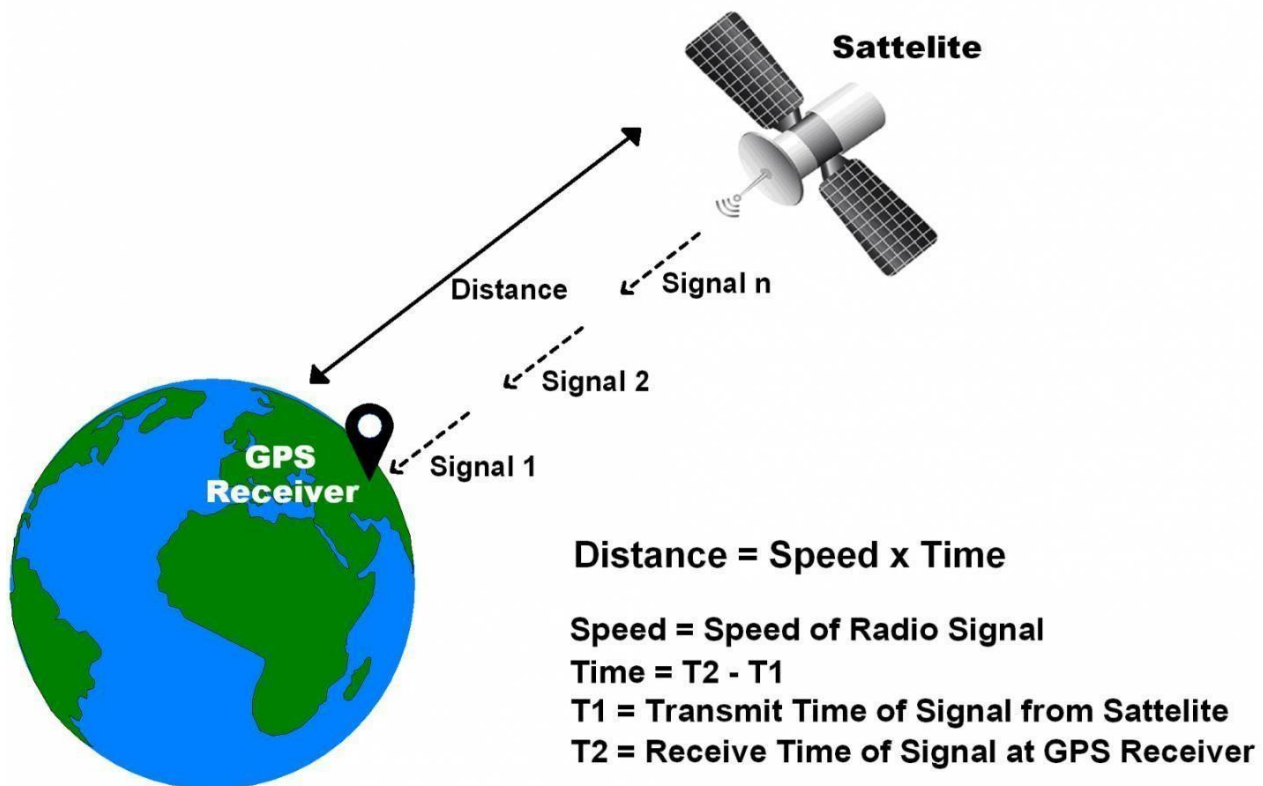


GIF Source: Wikipedia

GPS receiver uses a constellation of satellites and ground stations to calculate accurate location wherever it is located. These GPS satellites transmit information signal over radio frequency (1.1 to 1.5 GHz) to the receiver. With the help of this received information, a ground station or GPS module can compute its position and time.

How GPS Receiver Calculates its Position and Time

GPS receiver receives information signals from GPS satellites and calculates its distance from satellites. This is done by measuring the time required for the signal to travel from satellite to the receiver.



GPS Distance Calculation

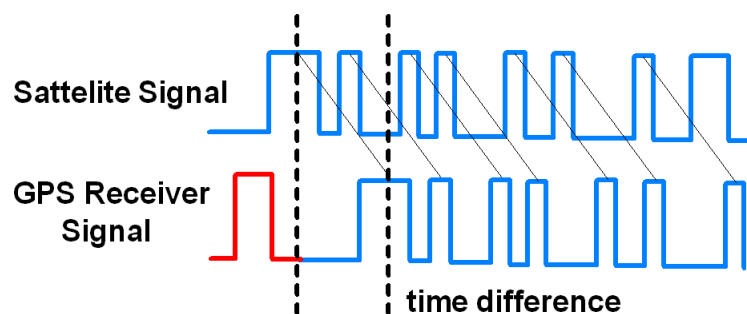
$$\text{Distance} = \text{Speed} \times \text{Time}$$

Where,

Speed = Speed of Radio signal which is approximately equal to the speed of light i.e. 3×10^8

Time = Time required for a signal to travel from the satellite to the receiver.

By subtracting the sent time from the received time, we can determine the travel time.



GPS Signal Time Difference

To determine distance, both the satellite and GPS receiver generate the same pseudocode signal at the same time. The satellite transmits the pseudocode; which is received by the GPS receiver. These two signals are compared and the difference between the signals is the travel time. Now, if the receiver knows the distance from 3 or more satellites and their location (which is sent by the satellites), then it can calculate its location by using [Trilateration](#) method.

GPS Module



GPS Receiver

GPS receiver module gives output in standard (National Marine Electronics Association) NMEA string format. It provides output serially on Tx pin with default 9600 Baud rate. This NMEA string output from GPS receiver contains different parameters separated by commas like longitude, latitude, altitude, time etc. Each string starts with '\$' and ends with carriage return/line feed sequence.

E.g.

```
$GPGGA,184237.000,1829.9639,N,07347.6174,E,1,05,2.1,607.1,M,-64.7,M,,0000*7D
```

```
$GPGSA,A,3,15,25,18,26,12,,,,,,,,,5.3,2.1,4.8*36
```

\$GPGSV,3,1,11,15,47,133,46,25,44,226,45,18,37,238,45,26,34,087,40*72

\$GPGSV,3,2,11,12,27,184,45,24,02,164,26,29,58,349,,05,26,034,*7F

\$GPGSV,3,3,11,21,25,303,,02,11,071,,22,01,228,*40

\$GPRMC,184237.000,A,1829.9639,N,07347.6174,E,0.05,180.19,230514,,,A*64



Pin Description

GPS Receiver Module

VCC: Power Supply 3.3 – 6 V

GND: Ground

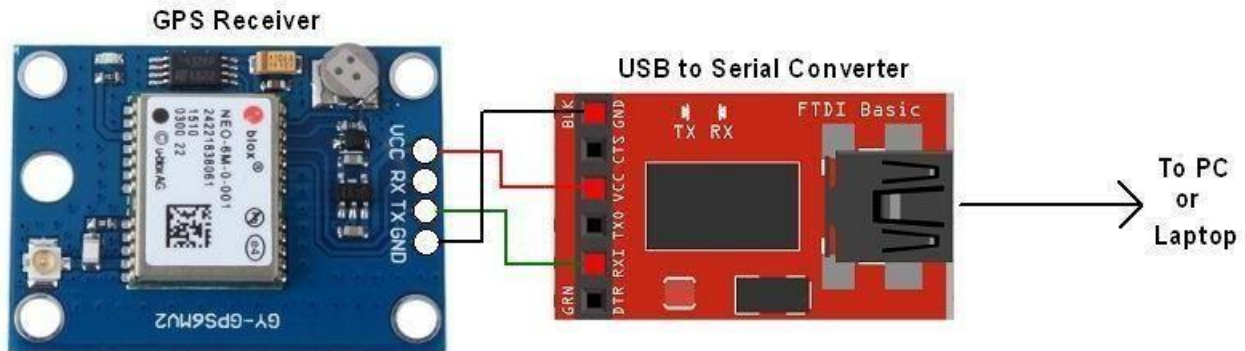
TX: Transmit data serially which gives information about location, time etc.

RX: Receive Data serially. It is required when we want to configure GPS module.

Check GPS module

Before Interfacing GPS module with PIC18F4550 microcontroller, we can check the output of GPS module. From that string, we can extract information like longitude, latitude, time which is helpful to find location and timing information.

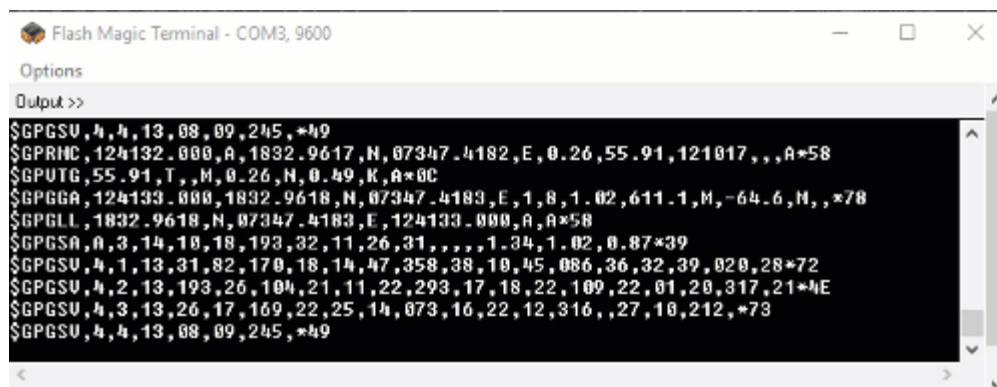
To do this, connect this GPS module to the PC via USB to Serial converter or DB9 connector. Also, it is necessary to keep antenna of GPS module on proper location.



GPS Serial Interface

1. Now open any serial terminal e.g. Realterm, Hyper terminal, Putty etc. on PC/laptop.
2. Open the PORT with 9600 baud rate.
3. The terminal will show data coming from GPS receiver module.

The output data from GPS receiver module displaying on a serial terminal as follows.



In the above string, the NMEA string starting with “\$GPGGA” is most popularly used. It provides us Time, Longitude, Latitude and Altitude along with directions. This information is helpful to find Time and Location.

E.g.

\$GPGGA,184241.000,1829.9639,N,07347.6174,E,1,05,2.1,607.1,M,-64.7,M,,0000*7C

Name	Example	Units	Description
Message ID	\$GPGGA		GGA Protocol Header
UTC Time	184241.000		hhmmss.sss
Latitude	1829.9639		ddmm.mmmm
N/S Indicator	N		N=North, S=South
Longitude	07347.6174		dddmm.mmmm
E/W Indicator	E		E=East, W=West
Position Fix Indicator	1		Fix GPS SPS mode
Satellites Used	05		Range 0 to 12
HDOP	2.1		Horizontal Dilution of Precision
MSL Altitude	607.1	Meters	Mean Sea Level
Units	M	Meters	
Geoid Separation	64.7	Meters	
Units	M	Meters	
Age of Diff. Corr.	-		Null field if DGPS is not used
Diff. Ref Station ID	0000		
Checksum	*7C		
Carriage return Line Feed	<CR><LF>		End of message transmission

CHAPTER-5

SOFTWARE DESCRIPTION

Arduino Software (IDE)

The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuine hardware to upload programs and communicate with them.

Writing Sketches

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension .ino. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

NB: Versions of the Arduino Software (IDE) prior to 1.0 saved sketches with the extension .pde. It is possible to open these files with version 1.0, you will be prompted to save the sketch with the .ino extension on save.

Verify

Checks your code for errors compiling it.

New

Creates a new sketch.

Open

Presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window overwriting its content.

Note: due to a bug in Java, this menu doesn't scroll; if you need to open a sketch late in the list use the File | Sketchbook menu instead.

Save

Saves your sketch.

Serial monitor

Opens the serial monitor

Additional commands are found within the five menus: File, Edit, Sketch, Tools, Help. The menus are context sensitive, which means only those items relevant to the work currently being carried out are available.

File

- New

Creates a new instance of the editor, with the bare minimum structure of a sketch already in place.

- Open

Allows to load a sketch file browsing through the computer drives and folders.

- OpenRecent

Provides a short list of the most recent sketches, ready to be opened.

- Sketchbook

Shows the current sketches within the sketchbook folder structure; clicking on any name opens the corresponding sketch in a new editor instance.

- Examples

Any example provided by the Arduino Software (IDE) or library shows up in this menu item.

All the examples are structured in a tree that allows easy access by topic or library.

- Close

Closes the instance of the Arduino Software from which it is clicked.

- Save

Saves the sketch with the current name. If the file hasn't been named before, a name will be provided in a "Save as.." window.

- Saveas...

Allows to save the current sketch with a different name.

- PageSetup

It shows the Page Setup window for printing.

- **Print**

Sends the current sketch to the printer according to the settings defined in Page Setup.

- **Preferences**

Opens the Preferences window where some settings of the IDE may be customized, as the language of the IDE interface.

- **Quit**

Closes all IDE windows. The same sketches open when Quit was chosen will be automatically reopened the next time you start the IDE.

Edit

- **Undo/Redo**

Goes back of one or more steps you did while editing; when you go back, you may go forward with Redo.

- **Cut**

Removes the selected text from the editor and places it into the clipboard.

- **Copy**

Duplicates the selected text in the editor and places it into the clipboard.

- **CopyforForum**

Copies the code of your sketch to the clipboard in a form suitable for posting to the forum, complete with syntax coloring.

- **CopyasHTML**

Copies the code of your sketch to the clipboard as HTML, suitable for embedding in web pages.

- **Paste**

Puts the contents of the clipboard at the cursor position, in the editor.

- **SelectAll**

Selects and highlights the whole content of the editor.

- **Comment/Uncomment**

Puts or removes the // comment marker at the beginning of each selected line.

- Increase/DecreaseIndent

Adds or subtracts a space at the beginning of each selected line, moving the text one space on the right or eliminating a space at the beginning.

- Find

Opens the Find and Replace window where you can specify text to search inside the current sketch according to several options.

- FindNext

Highlights the next occurrence - if any - of the string specified as the search item in the Find window, relative to the cursor position.

- FindPrevious

Highlights the previous occurrence - if any - of the string specified as the search item in the Find window relative to the cursor position.

Sketch

- Verify/Compile

Checks your sketch for errors compiling it; it will report memory usage for code and variables in the console area.

- Upload

Compiles and loads the binary file onto the configured board through the configured Port.

- UploadUsingProgrammer

This will overwrite the bootloader on the board; you will need to use Tools > Burn Bootloader to restore it and be able to Upload to USB serial port again. However, it allows you to use the full capacity of the Flash memory for your sketch. Please note that this command will NOT burn the fuses. To do so a Tools -> Burn Bootloader command must be executed.

ExportCompiledBinary

Saves a .hex file that may be kept as archive or sent to the board using other tools

Show sketch folder

- Opens the current sketch folder.
- IncludeLibrary

Adds a library to your sketch by inserting `#include` statements at the start of your code. For more details, see [libraries](#) below. Additionally, from this menu item you can access the Library Manager and import new libraries from .zip files.

- AddFile...

Adds a source file to the sketch (it will be copied from its current location). The new file appears in a new tab in the sketch window. Files can be removed from the sketch using the tab menu accessible clicking on the small triangle icon below the serial monitor one on the right side of the toolbar.

Tools

- AutoFormat

This formats your code nicely: i.e. indents it so that opening and closing curly braces line up, and that the statements inside curly braces are indented more.

- ArchiveSketch

Archives a copy of the current sketch in .zip format. The archive is placed in the same directory as the sketch.

- FixEncoding&Reload

Fixes possible discrepancies between the editor char map encoding and other operating systems char maps.

- SerialMonitor

Opens the serial monitor window and initiates the exchange of data with any connected board on the currently selected Port. This usually resets the board, if the board supports Reset over serial port opening.

- Board

Select the board that you're using. See below for [descriptions of the various boards](#).

- Port

This menu contains all the serial devices (real or virtual) on your machine. It should automatically refresh every time you open the top-level tools menu.

- Programmer

For selecting a hardware programmer when programming a board or chip and not using the onboard USB-serial connection. Normally you won't need this, but if you're burning a bootloader to a new microcontroller, you will use this.

- Burn

Bootloader

The items in this menu allow you to burn a bootloader onto the microcontroller on an Arduino board. This is not required for normal use of an Arduino or Genuino board but is useful if you purchase a new ATmega microcontroller (which normally come without a bootloader). Ensure that you've selected the correct board from the Boards menu before burning the bootloader on the target board. This command also set the right fuses.

Help

Here you find easy access to a number of documents that come with the Arduino Software (IDE). You have access to Getting Started, Reference, this guide to the IDE and other documents locally, without an internet connection. The documents are a local copy of the online ones and may link back to our online website.

- FindinReference

This is the only interactive function of the Help menu: it directly selects the relevant page in the local copy of the Reference for the function or command under the cursor.

Sketchbook

The Arduino Software (IDE) uses the concept of a sketchbook: a standard place to store your programs (or sketches). The sketches in your sketchbook can be opened from the File > Sketchbook menu or from the Open button on the toolbar. The first time you run the Arduino software, it will automatically create a directory for your sketchbook. You can view or change the location of the sketchbook location from with the Preferences dialog.

Beginning with version 1.0, files are saved with a .ino file extension. Previous versions use the .pde extension. You may still open .pde named files in version 1.0 and later, the software will automatically rename the extension to .ino.

Tabs, Multiple Files, and Compilation

Allows you to manage sketches with more than one file (each of which appears in its own tab). These can be normal Arduino code files (no visible extension), C files (.c extension), C++ files (.cpp), or header files (.h).

Uploading

Before uploading your sketch, you need to select the correct items from the Tools > Board and Tools > Port menus. The boards are described below. On the Mac, the serial port is probably something like /dev/tty.usbmodem241 (for an Uno or Mega2560 or Leonardo) or /dev/tty.usbserial-1B1 (for a Duemilanove or earlier USB board), or /dev/tty.USA19QW1b1P1.1 (for a serial board connected with a Keyspan USB-to-Serial adapter). On Windows, it's probably COM1 or COM2 (for a serial board) or COM4, COM5, COM7, or higher (for a USB board) - to find out, you look for USB serial device in the ports section of the Windows Device Manager. On Linux, it should be /dev/ttyACMx , /dev/ttyUSBx or similar. Once you've selected the correct serial port and board, press the upload button in the toolbar or select the Upload item from the Sketch menu. Current Arduino boards will reset automatically and begin the upload. With older boards (pre-Diecimila) that lack auto-reset, you'll need to press the reset button on the board just before starting the upload. On most boards, you'll see the RX and TX LEDs blink as the sketch is uploaded. The Arduino Software (IDE) will display a message when the upload is complete, or show an error.

When you upload a sketch, you're using the Arduino bootloader, a small program that has been loaded on to the microcontroller on your board. It allows you to upload code without using any additional hardware.

The bootloader is active for a few seconds when the board resets; then it starts whichever sketch was most recently uploaded to the microcontroller. The bootloader will blink the on-board (pin 13) LED when it starts (i.e. when the board resets).

Libraries

Libraries provide extra functionality for use in sketches, e.g. working with hardware or manipulating data. To use a library in a sketch, select it from the Sketch > Import Library menu. This will insert one or more `#include` statements at the top of the sketch and compile the library with your sketch. Because libraries are uploaded to the board with your sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete its `#include` statements from the top of your code.

There is a [list of libraries](#) in the reference. Some libraries are included with the [Arduino](#) software. Others can be downloaded from a variety of sources or through the Library Manager. Starting with version 1.0.5 of the IDE, you can import a library from a zip file and use it in an open sketch. See these [instructions for installing a third-party library](#).

To write your own library, see [this tutorial](#).

Serial Monitor

Displays serial data being sent from the [Arduino](#) or Genuino board (USB or serial board). To send data to the board, enter text and click on the "send" button or press enter. Choose the baud rate from the drop-down that matches the rate passed to `Serial.begin` in your sketch. Note that on Windows, Mac or Linux, the [Arduino](#) or Genuino board will reset (rerun your sketch execution to the beginning) when you connect with the serial monitor. You can also talk to the board from Processing, Flash, MaxMSP, etc (see the [interfacing page](#) for details)

Preferences

Some preferences can be set in the preferences dialog (found under the Arduino menu on the Mac, or File on Windows and Linux). The rest can be found in the preferences file, whose location is shown in the preference dialog.

Language Support



Since version 1.0.1 , the Arduino Software (IDE) has been translated into 30+ different languages. By default, the IDE loads in the language selected by your operating system. (Note: on Windows and possibly Linux, this is determined by the locale setting which controls currency and date formats, not by the language the operating system is displayed in.)

If you would like to change the language manually, start the Arduino Software (IDE) and open the Preferences window. Next to the Editor Language there is a dropdown menu of currently supported languages. Select your preferred language from the menu, and restart the software to use the selected language. If your operating system language is not supported, the Arduino Software (IDE) will default to English.

You can return the software to its default setting of selecting its language based on your operating system by selecting System Default from the Editor Language drop-down. This setting will take effect when you restart the Arduino Software (IDE). Similarly, after changing your operating system's settings, you must restart the Arduino Software (IDE) to update it to the new default language.

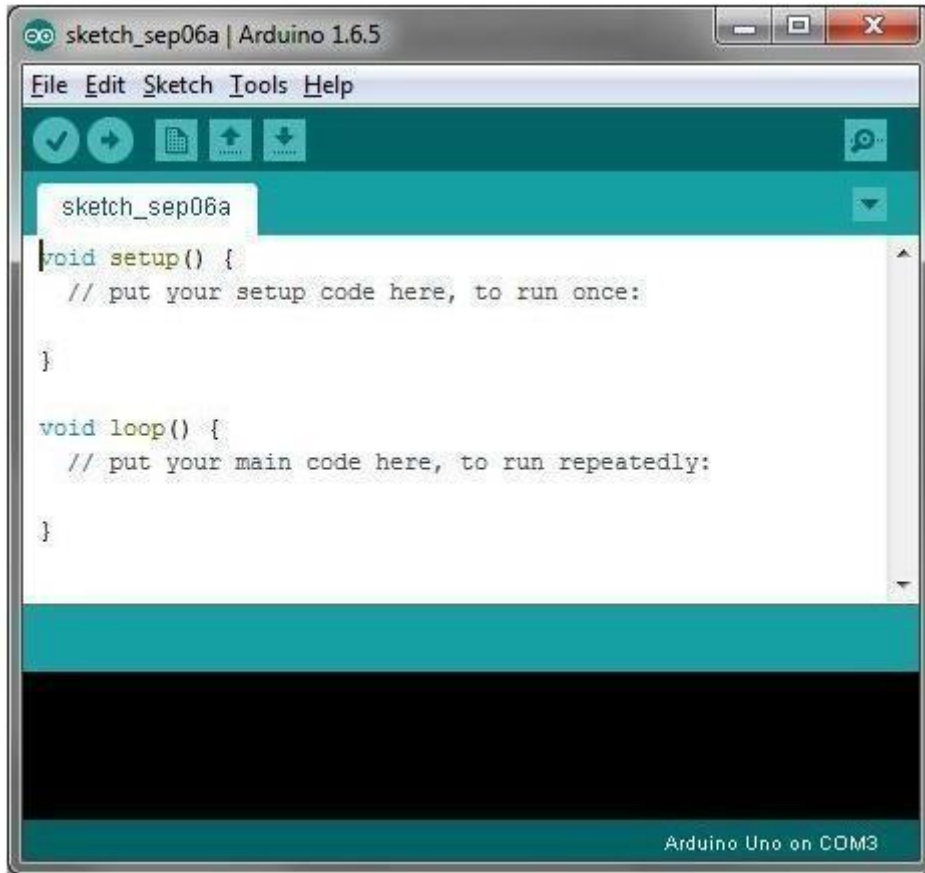
Boards

The board selection has two effects: it sets the parameters (e.g. CPU speed and baud rate) used when compiling and uploading sketches; and sets the file and fuse settings used by the burn bootloader command. Some of the board definitions differ only in the latter, so even if you've been uploading successfully with a particular selection you'll want to check it before burning the bootloader. You can find a comparison table between the various boards [here](#).

Arduino Software (IDE) includes the built in support for the boards in the following list, all based on the AVR Core. The Boards Manager included in the standard installation allows to add support for the growing number of new boards based on different cores like Arduino Due, Arduino Zero, Edison, Galileo and so on.

Arduino IDE: Initial Setup

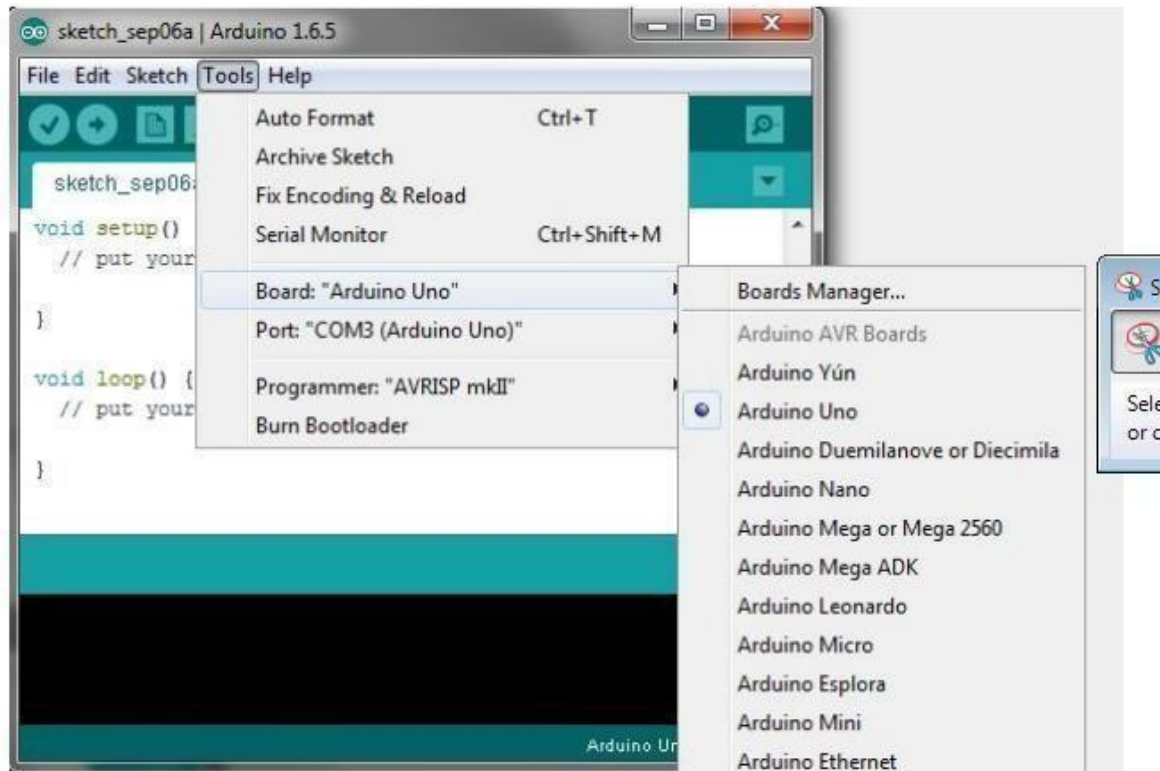
Download Arduino Integrated Design Environment (IDE) here (Most recent version: 1.6.5): <https://www.arduino.cc/en/Main/Software> This is the Arduino IDE once it's been opened. It opens into a blank sketch where you can start programming immediately. First, we should configure the board and port settings to allow us to upload code. Connect your Arduino board to the PC via the USB cable.



Arduino IDE Default Window

IDE: Board Setup

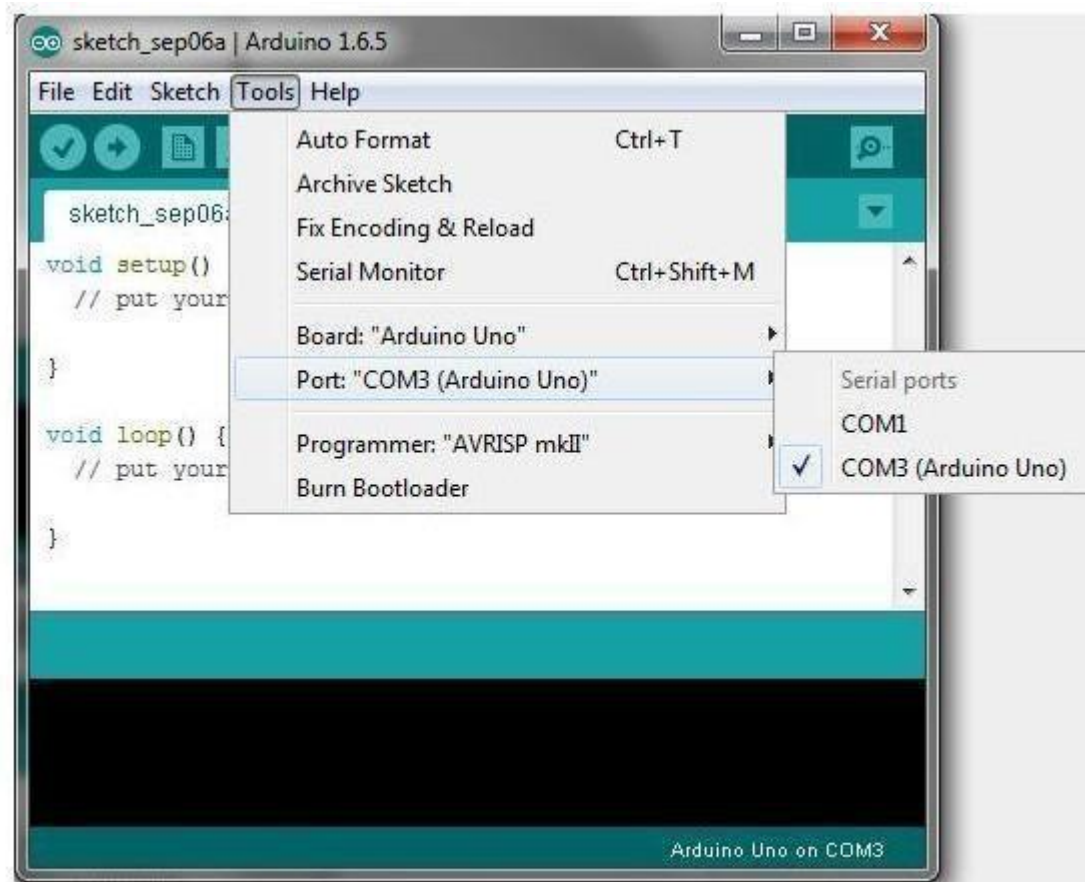
You have to tell the Arduino IDE what board you are uploading to. Select the Tools pulldown menu and go to Board. This list is populated by default with the currently available Arduino Boards that are developed by Arduino. If you are using an Uno or an Uno-Compatible Clone (ex. Funduino, SainSmart, IEIK, etc.), select Arduino Uno. If you are using another board/clone, select that board.



Arduino IDE: Board Setup Procedure

IDE: COM Port Setup

If you downloaded the Arduino IDE before plugging in your Arduino board, when you plugged in the board, the USB drivers should have installed automatically. The most recent Arduino IDE should recognize connected boards and label them with which COM port they are using. Select the Tools pulldown menu and then Port. Here it should list all open COM ports, and if there is a recognized Arduino Board, it will also give it's name. Select the Arduino board that you have connected to the PC. If the setup was successful, in the bottom right of the Arduino IDE, you should see the board type and COM number of the board you plan to program. Note: the Arduino Uno occupies the next available COM port; it will not always be COM3.



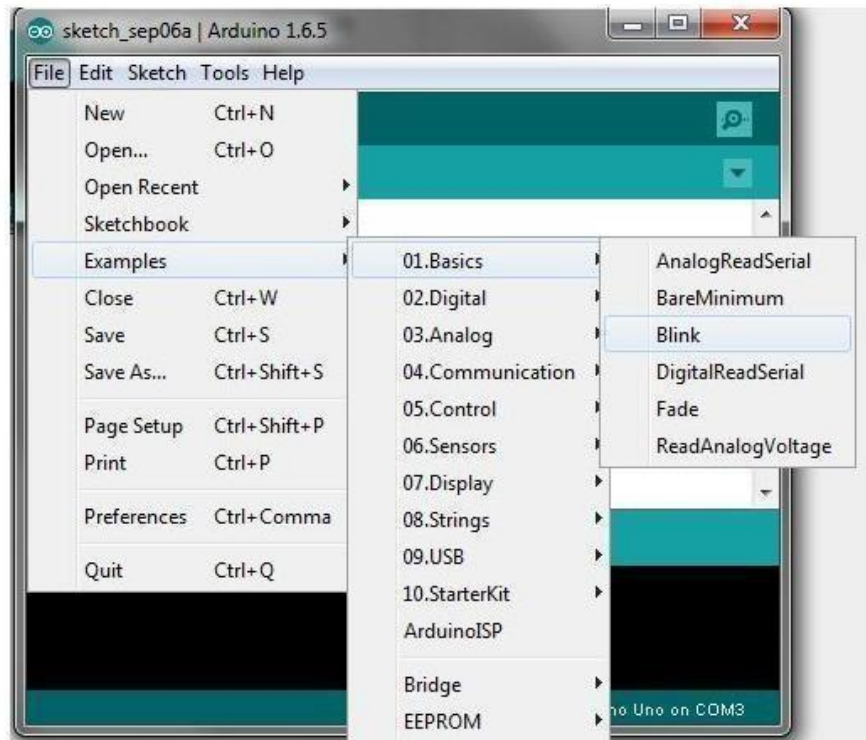
Arduino IDE: COM Port Setup

At this point, your board should be set up for programming, and you can begin writing and uploading code.

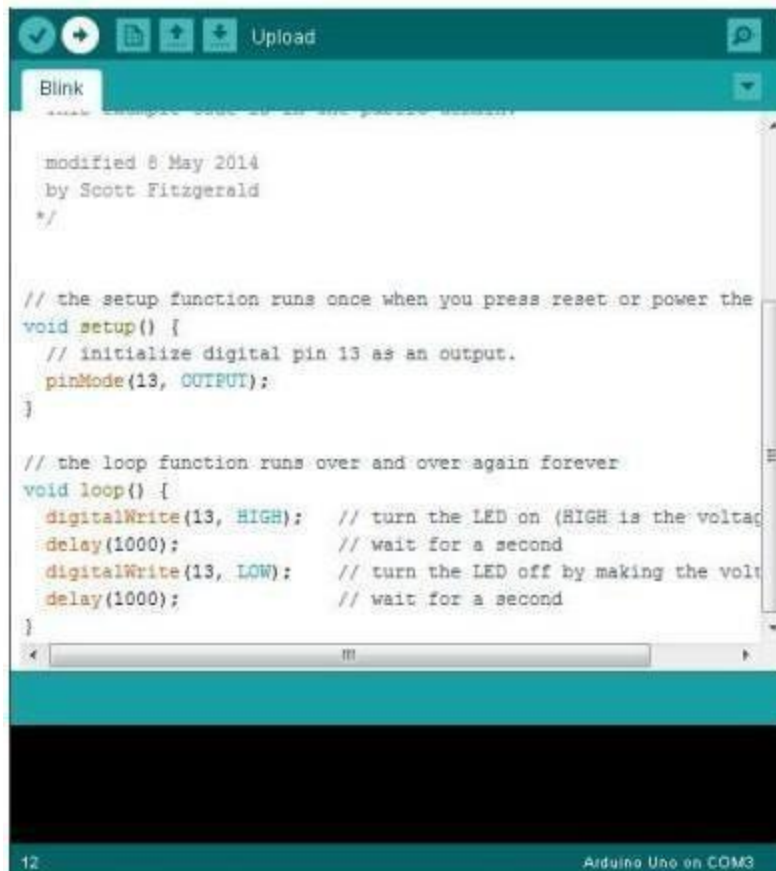
Testing Your Settings: Uploading Blink

One common procedure to test whether the board you are using is properly set up is to upload the “Blink” sketch. This sketch is included with all Arduino IDE releases and can be accessed by the File pull-down menu and going to Examples, 01.Basics, and then select Blink. Standard Arduino Boards include a surface-mounted LED labeled “L” or “LED” next to the “RX” and “TX” LEDs, that is connected to digital pin 13. This sketch will blink the LED at a regular interval, and is an easy way to confirm if your board is set up properly and you were successful in uploading code. Open the “Blink” sketch and press the “Upload” button in the upper-left corner to upload “Blink” to the board.

Upload Button:



Arduino IDE: Loading Blink Sketch



Arduino IDE: Uploading Blink

Guide Summary:

1. Download and install Arduino IDE (<https://www.arduino.cc/en/Main/Software>)
2. Plug in your Arduino Board
3. Select the proper board in the IDE (Tools>Boards>Arduino Uno)
4. Select the proper COM port (Tools>Port>COMx (Arduino Uno))
5. Open the “Blink” sketch (File>Examples>Basics>01.Blink)
6. Press the Upload button to upload the program to the board
7. Confirm that your board is working as expected by observing LED

Troubleshooting Uploading Errors:

Arduino has lots of community support and documentation. Your best bet when running into unexpected problems is to search online for help. You should be able to find a forum where someone had the same problem you are having, and someone helped them fix it. If you don't find results, try modifying your search, or post on the Arduino forums.

- My board isn't listed under devices and is not recognized by IDE:
 - Most likely, this means that the ATmega328p chip is not programmed with the Arduino firmware. If you have a separate working Uno available, you can program the unprogrammed chip using this guide and a few jumper cables: <https://www.arduino.cc/en/Tutorial/ArduinoISP>
 - If you don't have a separate Arduino available, let me know and I can use an Atmel Programmer to upload the firmware.
 - There may be hardware damage if you had the board plugged into USB and external power at the same time. You may have to replace the chip if this is the case.
- Error Message: avrdude: stk500_recv(): programmer is not responding
 - Double-check that you are using the correct COM port.
 - Make sure that your Arduino Board is plugged into the computer.
- The IDE says “Uploading...” after pressing the upload button, but nothing is happening.
 - Double-check that you have the correct board selected in the Tools menu.
 - Depending on the size of your program, it may take a few seconds to upload. If you feel like it is taking too long, it may be encountering an error and you can try unplugging and plugging in the Arduino board

CHAPTER 6

CONCLUSION :

The IoT Based Automatic Vehicle Accident Detection and Rescue System was successfully designed and implemented to detect vehicle accidents automatically and provide immediate emergency alerts. The system integrates sensors such as an accelerometer/vibration sensor, GPS module, GSM/Wi-Fi module, and a microcontroller to monitor vehicle movement and detect abnormal impacts. When an accident occurs, the system instantly captures the exact geographical location using GPS and sends an alert message to emergency contacts and rescue services through GSM or internet communication. This reduces the time taken to inform authorities compared to manual reporting and increases the chances of saving lives. The project demonstrates that IoT technology can play a significant role in improving road safety and emergency response systems. The prototype showed reliable accident detection in testing conditions, quick message transmission, and accurate location tracking. Although the system depends on network availability and proper sensor calibration, it proves to be an effective, low-cost, and scalable solution for real-time accident monitoring. In conclusion, the proposed system enhances road safety by minimizing response delay and providing automated rescue support, thereby contributing to smarter and safer transportation systems.

FUTURE ENHANCEMENTS :

Although the proposed system works effectively, it can be further improved with the following enhancements:

1. Integration with Mobile Application

A dedicated mobile app can be developed to:

- Provide real-time accident alerts

- Track vehicle location live

- Allow emergency contacts to monitor status

- Cancel false alarms easily

2. AI-Based Accident Detection

Artificial Intelligence and Machine Learning can be added to:

- Reduce false accident detection

- Analyze vehicle speed and impact severity

- Predict accident risk in advance

3. Automatic Communication with Nearby Hospitals

The system can be connected to:

- Nearby hospitals

- Ambulance services

- Police control rooms

This ensures faster emergency response without manual intervention.

CHAPTER 7

REFERENCES

- ▶ [1] Kiran Sawant, Imran Bhole, Prashant Kokane, Piraji Doiphode, Prof. Yogesh Thorat, “Accident Alert and Vehicle Tracking System”, International Journal of Innovative Analysis in laptop and Communication Engineering, Vol. 4, Issue 5, May 2016.
- ▶ [2] Mrs Manasi Patil, Aanchal Rawat, Prateek Singh, Srishtie Dixit, “Accident Detection and Ambulance Control using Intelligent Traffic Control System”, International Journal of Engineering Trends and Technology (IJETT), Volume 34-Number 8, April 2016.
- ▶ [3] V.Sagar Reddy, Dr.L.PadmaSree, V. Naveen Kumar, “Design and Development of accelerometer-based System for driver safety”, International Journal of Science, Engineering and Technology Research (IJSETR), Volume 3, Issue 12, December 2014.
- ▶ [4] Sri Krishna Chaitanya Varma, Poornesh, Tarun Varma, Harsha, “Automatic Vehicle Accident Detection and Messaging System Using GPS and GSM Modems”, International Journal of Scientific & Engineering Research, Volume 4, Issue 8, August 2013.
- ▶ [5] Apurva Mane, Jaideep Rana, “Vehicle Collision detection and Remote Alarm Device using Arduino”, International Journal of Current Engineering and Technology, Vol.4, No.3, June 2014.
- ▶ [6] Prof.Mrs.Bhagya Lakshmi V, Prof.Savitha Hiremath, Prof.Sanjeev Mhamane, “FPGA Based Vehicle Tracking and Accident Warning using GPS”, International Journal of Scientific & Engineering Research, Volume 5, Issue 2, February-2014.

